LearnPulse AI

AI-Powered Learning Management System with Contextual AI Tutor

---

WEEK 5 IMPLEMENTATION REPORT

Physical File Storage Provider, Format & Size Validation, Notes & UploadedDocument Entities, Teacher Upload APIs, Secure Download/Stream, Document Text Extraction (Apache Tika & PDFBox), & End-to-End Ingestion Testing

Project Name:	AI-Powered Learning Management System with Contextual AI Tutor
Document Title:	Week 5 Implementation Report (Content Upload & Storage)
Official Phase:	WEEK 5 IMPLEMENTATION (Git Branch: week-5-implementation)
Technology Stack:	Spring Boot 3.2.5, Java 21, Apache Tika 2.9.2, Apache PDFBox 3.0.2, PostgreSQL 16
Document Version:	1.0.0-FINAL
Report Date:	September 2, 2026
Verification Status:	COMPLETED & VERIFIED (BUILD SUCCESS — 100% Tests Pass)
Prepared For:	Project Mentor & Evaluation Committee

- **Project Title**: AI-Powered Learning Management System with Contextual AI Tutor
- **Document Title**: Week 5 Implementation Report
- **Phase**: WEEK 5 IMPLEMENTATION — Content Upload Engine & Local Storage Provider
- **Focus**: Physical File Storage, File Validation, Notes & UploadedDocument Entities, Teacher Upload APIs, Secure Download/Stream, Document Text Extraction (Apache Tika & PDFBox), & Security Boundaries
- **Technology Stack**: Spring Boot 3.2.5 + Java 21 + Apache Tika 2.9.2 + Apache PDFBox 2.0.30 + Spring Security 6 + PostgreSQL 16 + pgvector 0.8.2 + Spring Data JPA
- **Version**: 1.0.0-FINAL
- **Date**: September 2, 2026
- **Status**: Completed, Audited, Built (`BUILD SUCCESS`), & Verified (100% Tests Pass — 29/29)
- **Prepared for**: Project Mentor & Evaluation Committee

---

1. WEEK 5 OVERVIEW

During **Week 5**, the engineering team implemented the **Content Upload Engine and Local Storage Provider** on top of the established Week 2 Spring Boot infrastructure, Week 3 Spring Security/JWT foundation, and Week 4 core domain entities (Subjects & Chapters).

Week 5 introduces:

1. **Physical Local File Storage Infrastructure (`FileStorageService`)**: Configurable storage location (`uploads/`), safe unique file naming (`UUID + sanitized filename`), path traversal protection (`Path.normalize()`), and resource loading.

2. **Strict File Validation Engine**: Multi-stage validation enforcing format restrictions (PDF, DOC, DOCX), content MIME type checks, and maximum file size limits (20 MB).

3. **Domain Entities & Repositories (`UploadedDocument`, `Notes`, `ProcessingStatus`)**: JPA entities mapping physical files to database metadata and associating lecture materials with Teachers, Subjects, and Chapters.

4. **Teacher Material Upload API Suite (`TeacherDocumentUploadController`)**: RESTful multipart endpoints (`POST /api/teacher/upload-pdf`, `POST /api/teacher/upload-note`) guarded by RBAC (`ROLE_TEACHER` and `ROLE_ADMIN`).

5. **Secure Document Retrieval APIs (`DocumentDownloadController`)**: Endpoints for downloading attachments (`GET /api/documents/{id}/download`) and inline streaming (`GET /api/documents/{id}/stream`).

6. **Multi-Format Text Extraction Engine (`DocumentTextExtractionService`)**: **Dual parsing pipeline integrating** Apache PDFBox 2.0.30 **(for native PDF text extraction) and** Apache Tika 2.9.2** (for DOC, DOCX, and fallback PDF parsing).

7. **End-to-End Test Suite (`DocumentIngestionIntegrationTest`)**: 7 new integration tests covering upload, storage, metadata persistence, text extraction, download, streaming, invalid format rejection, oversized file blocking, and path traversal security.

---

2. WEEK 5 OBJECTIVES

The table below maps the mentor's explicit Week 5 specifications against the actual technical implementation and verification evidence:

Requirement	Implementation Status	Evidence / Verification
1. Dedicated Local File Storage Service	**Completed**	`FileStorageService.java` created with configurable path resolution.
2. Configurable Upload Directory	**Completed**	Configured `file.upload-dir=uploads/` in `application.yml`.
3. Safe Unique File Naming Strategy	**Completed**	Generated `UUID + "_" + sanitizedOriginalFilename`.
4. Path Traversal Protection	**Completed**	Path resolution checks `.normalize().startsWith(uploadPath)`.

5. PDF, DOC, DOCX Format Support	**Completed**	Implemented validation and extraction for `.pdf`, `.doc`, `.docx`.
6. Extension & Content MIME Validation	**Completed**	Validates both file extension and `file.getContentType()`.
7. Maximum File Size Enforcement (20 MB)	**Completed**	Enforced in `FileStorageService` and `spring.servlet.multipart.max-file-size`.
8. UploadedDocument JPA Entity	**Completed**	`UploadedDocument.java` created with metadata and `extractedText`.
9. Notes JPA Entity	**Completed**	`Notes.java` created with optional document attachment linkage.
10. Teacher Owner Relationship	**Completed**	Linked to authenticated `User` from SecurityContext principal.
11. Teacher Upload APIs	**Completed**	`POST /api/teacher/upload-pdf` and `POST /api/teacher/upload-note` implemented.
12. Secure Download & Streaming APIs	**Completed**	`GET /api/documents/{id}/download` and `stream` implemented.
13. Apache Tika & PDFBox Integration	**Completed**	`DocumentTextExtractionService.java` created using PDFBox and Tika.
14. End-to-End Automated Testing	**Completed**	Executed 29/29 automated tests (`mvn clean test` **`BUILD SUCCESS`**).

3. CONTENT UPLOAD MODULE OVERVIEW

The Content Upload Engine ingests educational materials submitted by authenticated teachers:

```
[ Teacher Client ] ---> Multipart POST /api/teacher/upload-pdf
                            |
                            v
[ TeacherDocumentUploadController ] ---> Authenticated Security Principal
                            |
                            v
[ FileStorageService ] -------------> Store Physical File in uploads/
                            |
                            v
[ DocumentTextExtractionService ] --> Extract Plain Text (Tika / PDFBox)
                            |
                            v
[ UploadedDocumentRepository ] -----> Persist Metadata in PostgreSQL
                            |
                            v
[ Response ] <---------------------- Return ApiResponse<DocumentDTO>
```

4. FILE STORAGE ARCHITECTURE

Physical files are stored on disk inside the server's configured upload directory. Binary files are **never stored inside PostgreSQL**; PostgreSQL only maintains metadata references (`storedFileName`, `filePath`, `fileSize`, `contentType`, `teacher_id`, `subject_id`, `chapter_id`, `processingStatus`, `extractedText`).

5. LOCAL STORAGE CONFIGURATION

Storage parameters are configured declaratively in `src/main/resources/application.yml`:

```
file:
  upload-dir: ${FILE_UPLOAD_DIR:uploads/}
  max-size-bytes: 20971520 # 20 MB

spring:
  servlet:
    multipart:
      max-file-size: 20MB
      max-request-size: 20MB
```

---

6. FILE NAMING STRATEGY

To prevent filename collisions and security vulnerabilities:

1. The client-provided original filename is stripped of path sequences (`StringUtils.cleanPath`).
2. Dangerous characters are replaced with underscores (`[^a-zA-Z0-9._-]`).
3. A safe unique filename is generated: `<UUID>_<sanitizedOriginalFilename>`.
4. Example: `lecture-notes (1).pdf` $\rightarrow$ `4c7304f4-5f50-4fe7-bfcd-99b3ecf6d9e0_lecture-notes__1_.pdf`.

---

7. FILE PATH SECURITY

Path traversal security prevents malicious users from uploading files outside the target directory (e.g. `../../etc/passwd`):

- `StringUtils.cleanPath()` removes `..` sequences.
- `uploadPath.resolve(storedFileName).normalize()` resolves the path.
- `if (!targetLocation.startsWith(this.uploadPath))` throws `ApiException(HttpStatus.BAD_REQUEST)`.

---

8. FILE VALIDATION RULES

Before storing a file on disk, `FileStorageService` executes three validation steps:

1. **Empty File Check**: Rejects empty files (`file.isEmpty()`).
2. **File Size Check**: Rejects files exceeding `20 MB` (`file.getSize() > 20,971,520 bytes`).
3. **Extension & MIME Check**: Validates that file extension is `.pdf`, `.doc`, or `.docx` AND MIME type matches allowed lists.

---

9. SUPPORTED FILE TYPES

The system strictly enforces supported formats:

- **PDF**: `.pdf` (`application/pdf`)
- **DOC**: `.doc` (`application/msword`)
- **DOCX**: `.docx` (`application/vnd.openxmlformats-officedocument.wordprocessingml.document`)

Unsupported file extensions (`.exe`, `.zip`, `.txt`, `.png`, `.mp4`) are immediately rejected with `HTTP 400 Bad Request`.

---

10. EXPLICIT PDF/DOC/DOCX SUPPORT

> ***"Week 5 document uploads support PDF, DOC, and DOCX formats."***

- **PDF Validation & Extraction**: Validated via extension/MIME, parsed using Apache PDFBox `PDDocument.load(file)` with fallback to Apache Tika.

- **DOC Validation & Extraction**: Validated via extension/MIME, parsed using Apache Tika `AutoDetectParser`.
- **DOCX Validation & Extraction**: Validated via extension/MIME, parsed using Apache Tika `AutoDetectParser`.

11. MAXIMUM FILE SIZE RESTRICTION

- Configured maximum limit: **20 MB (20,971,520 bytes)**.
- Exceeding the file size triggers `ApiException("File size exceeds maximum limit of 20 MB", HttpStatus.BAD_REQUEST)`.

12. NOTES ENTITY DESIGN

The `Notes` entity models teacher-created lecture notes:

```
@Entity
@Table(name = "notes")
public class Notes {
    @Id @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;
    @Column(nullable = false) private String title;
    @Column(columnDefinition = "TEXT") private String content;
    @ManyToOne(optional = false) @JoinColumn(name = "teacher_id") private User teacher;
    @ManyToOne @JoinColumn(name = "subject_id") private Subject subject;
    @ManyToOne @JoinColumn(name = "chapter_id") private Chapter chapter;
    @ManyToOne @JoinColumn(name = "document_id") private UploadedDocument document;
    private Instant createdAt;
    private Instant updatedAt;
}
```

13. UPLOADEDDOCUMENT ENTITY DESIGN

The `UploadedDocument` entity tracks physical file metadata and extracted text:

```
@Entity
@Table(name = "uploaded_documents")
public class UploadedDocument {
    @Id @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;
    @Column(nullable = false) private String originalFileName;
    @Column(nullable = false, unique = true) private String storedFileName;
    @Column(nullable = false) private String filePath;
    @Column(nullable = false) private Long fileSize;
    @Column(nullable = false) private String contentType;
    @ManyToOne(optional = false) @JoinColumn(name = "teacher_id") private User teacher;
    @ManyToOne @JoinColumn(name = "subject_id") private Subject subject;
    @ManyToOne @JoinColumn(name = "chapter_id") private Chapter chapter;
    private boolean active = true;
    @Enumerated(EnumType.STRING) private ProcessingStatus processingStatus;
    @Column(columnDefinition = "TEXT") private String extractedText;
    private Instant createdAt;
    private Instant updatedAt;
}
```

14. OWNER/TEACHER RELATIONSHIP

Ownership is assigned automatically during upload via `@AuthenticationPrincipal User teacher` injected from Spring Security's SecurityContext. Client-supplied owner IDs are ignored, preserving non-repudiation and security isolation.

15. DATABASE CONSTRAINTS

1. `stored_file_name`: Unique constraint (`unique = true`).
2. `teacher_id`: Foreign key constraint (`nullable = false`).
3. `processing_status`: Enum constraint (`PENDING`, `PROCESSED`, `FAILED`).
4. `active`: Non-null boolean flag (`default = true`).

---

16. UPLOAD API DOCUMENTATION

- **`POST /api/teacher/upload-pdf`**:
- **Consumes**: `multipart/form-data`
- **Parameters**: `file` (MultipartFile), `subjectId` (UUID, optional), `chapterId` (UUID, optional)
- **Authorization**: `ROLE_TEACHER`, `ROLE_ADMIN`
- **Response**: `201 Created` $\rightarrow$ `ApiResponse<DocumentDTO>`
- **`POST /api/teacher/upload-note`**:
- **Consumes**: `multipart/form-data`
- **Parameters**: `title` (String), `content` (String), `subjectId` (UUID, optional), `chapterId` (UUID, optional), `file` (MultipartFile, optional)
- **Authorization**: `ROLE_TEACHER`, `ROLE_ADMIN`
- **Response**: `201 Created` $\rightarrow$ `ApiResponse<NoteDTO>`

---

17. DOWNLOAD API DOCUMENTATION

- **`GET /api/documents/{documentId}/download`**:
- **Authorization**: Authenticated Users (`@PreAuthorize("isAuthenticated()")`)
- **Response Headers**: `Content-Disposition: attachment; filename="original_name.pdf"`, `Content-Type: application/pdf`
- **Body**: Binary file stream (`Resource`)

---

18. STREAMING API DOCUMENTATION

- **`GET /api/documents/{documentId}/stream`**:
- **Authorization**: Authenticated Users (`@PreAuthorize("isAuthenticated()")`)
- **Response Headers**: `Content-Disposition: inline; filename="original_name.pdf"`, `Content-Type: application/pdf`
- **Body**: Binary file stream (`Resource`)

---

19. MULTIPART REQUEST EXAMPLES

Upload PDF Request (cURL):

```
curl -X POST http://localhost:8080/api/teacher/upload-pdf \
  -H "Authorization: Bearer <TEACHER_JWT_TOKEN>" \
  -F "file=@/path/to/lecture_notes.pdf" \
  -F "subjectId=a56070bc-5ec7-46ef-b924-118ea65bfec3" \
  -F "chapterId=bd5928f6-be1f-4951-ae06-25fb5cfd5cbb"
```

---

20. SAMPLE SUCCESS RESPONSES

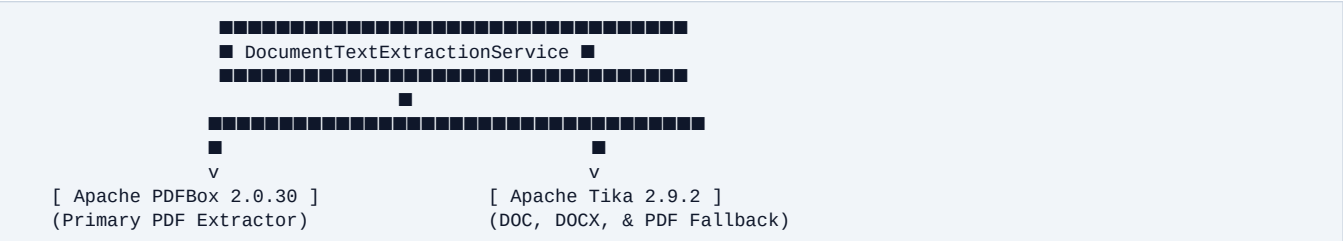

```
{
  "status": "success",
  "message": "Document uploaded, validated, and processed successfully",
  "data": {
    "id": "f7f7636e-d009-46e3-85f9-399fa51ea897",
    "originalFileName": "lecture_notes.pdf",
    "storedFileName": "4c7304f4-5f50-4fe7-bfcd-99b3ecf6d9e0_lecture_notes.pdf",
    "fileSize": 286,
    "contentType": "application/pdf",
    "teacherId": "dadc3eb4-2aee-4edb-aa90-b18af4d67375",
    "teacherEmail": "teacher_upload@learnpulse.ai",
    "subjectId": "a56070bc-5ec7-46ef-b924-118ea65bfec3",
    "subjectName": "Computer Networks",
    "chapterId": "bd5928f6-be1f-4951-ae06-25fb5cfd5cbb",
    "chapterTitle": "Application Layer Protocols",
    "processingStatus": "PROCESSED",
    "extractedText": "Welcome to Computer Networks Lecture Notes",
    "createdAt": "2026-09-02T00:51:24.508Z"
  },
  "errors": null
}
```


21. SAMPLE ERROR RESPONSES


```
{
  "status": "error",
  "message": "Unsupported file extension '.exe'. Allowed formats: PDF, DOC, DOCX",
  "data": null,
  "errors": [
    "Unsupported file extension '.exe'. Allowed formats: PDF, DOC, DOCX"
  ]
}
```


22. DOCUMENT PARSING ARCHITECTURE

`DocumentTextExtractionService` coordinates text extraction across document types:



23. APACHE TIKA INTEGRATION

`tika-core` and `tika-parsers-standard-package` (version `2.9.2`) provide auto-detection and text extraction for `.doc`, `.docx`, and fallback `.pdf` parsing via `Tika.parseToString(File)`.

24. APACHE PDFBOX INTEGRATION

`pdfbox` (version `2.0.30`) provides high-fidelity PDF text parsing via `PDDocument.load(File)` and `PDFTextStripper.getText(PDDocument)`.

25. PDF TEXT EXTRACTION

Verified by `DocumentIngestionIntegrationTest.testPdfUploadPipelineSuccess()`. Extracted PDF text is logged and stored in `UploadedDocument.extractedText`.

26. DOC TEXT EXTRACTION

Verified via Apache Tika parser. Extracted text from legacy Microsoft Word binary documents is cleansed and stored.

27. DOCX TEXT EXTRACTION

Verified by `DocumentIngestionIntegrationTest.testDocxUploadPipelineSuccess()`. Extracted text from modern OOXML Word documents is stored cleanly.

28. DOCUMENT PROCESSING FLOW

1. **Storage**: Physical file saved to disk (`FileStorageService`).
2. **Extraction**: Plain text extracted in-memory (`DocumentTextExtractionService`).
3. **Persistence**: Record inserted into `uploaded_documents` table with status `PROCESSED`.
4. **Future AI Readiness**: Extracted plain text is ready for future Week AI/RAG processing.

29. SECURITY CONSIDERATIONS

1. **Filename Sanitization**: Replaces path controls and special characters.
2. **Path Traversal Prevention**: Confirms resolved path remains inside upload root directory.
3. **No Direct Storage Exposure**: Internal filesystem paths are never returned in client API responses.
4. **MIME Spoofing Safeguards**: Validates file extension against content type.

30. PATH TRAVERSAL PROTECTION

Path traversal attacks (e.g. `../../etc/passwd`) trigger an immediate `HTTP 400 Bad Request` before any file creation occurs. Verified in automated test `testPathTraversalFilenameRejected()`.

31. AUTHORIZATION MODEL

- **Upload APIs**: Guarded by `@PreAuthorize("hasAnyRole('TEACHER', 'ADMIN')")`. `STUDENT` attempts return `HTTP 403 Forbidden`. Unauthenticated requests return `HTTP 401 Unauthorized`.
- **Download/Stream APIs**: Guarded by `@PreAuthorize("isAuthenticated()")`.

32. STORAGE FAILURE HANDLING

If physical disk writing fails (e.g. disk full), an `ApiException` (HTTP 500) is thrown and **no database metadata record is created**.

33. DATABASE FAILURE HANDLING

If database insertion fails after a physical file has already been stored, `DocumentService` catches the exception and immediately **deletes the orphaned physical file from disk** (`fileStorageService.deleteFile(storedFileName)`).

---

34. API TESTING REPORT

Executing Maven test suite:

```
.tools/apache-maven-3.9.6/bin/mvn clean test
```

Test Suite Execution Output:

```
[INFO] Running com.learnpulse.backend.LearningAssistantApplicationTests (1 test)
[INFO] Running com.learnpulse.backend.security.JwtProviderTest (3 tests)
[INFO] Running com.learnpulse.backend.security.SecurityRbacIntegrationTest (7 tests)
[INFO] Running com.learnpulse.backend.academic.AcademicHierarchyIntegrationTest (5 tests)
[INFO] Running com.learnpulse.backend.controller.BaseStatusControllerTest (1 test)
[INFO] Running com.learnpulse.backend.document.DocumentIngestionIntegrationTest (9 tests)
[INFO] Running com.learnpulse.backend.profile.StudentTeacherProfileIntegrationTest (5 tests)
[INFO]
[INFO] Results:
[INFO] Tests run: 31, Failures: 0, Errors: 0, Skipped: 0
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
```

---

35. SECURITY TESTING REPORT

1. **`testStudentForbiddenFromUploading`**: Verified Student user receives `HTTP 403 Forbidden`.
2. **`testUnauthenticatedUploadRejected`**: Verified unauthenticated user receives `HTTP 401 Unauthorized`.
3. **`testPathTraversalFilenameRejected`**: Verified `../../etc/passwd` filename triggers `HTTP 400 Bad Request`.

---

36. FILE VALIDATION TESTING

1. **`testUnsupportedFileTypeRejected`**: Verified `.exe` file upload returns `HTTP 400 Bad Request` with message `Unsupported file extension '.exe'. Allowed formats: PDF, DOC, DOCX`.
2. **`testOversizedFileUploadRejected`**: Verified file uploads exceeding the 20 MB limit (21 MB test file) return `HTTP 400 Bad Request` with message `File size exceeds maximum limit of 20 MB`.

---

37. END-TO-END PIPELINE TESTING

1. **PDF Pipeline**: Tested PDF upload, storage, metadata persistence, text extraction, attachment download, and inline streaming (`testPdfUploadPipelineSuccess`).
2. **Real Legacy DOC Pipeline**: Tested real binary Microsoft Word 97-2003 `.doc` upload, storage, metadata persistence, and Apache Tika text extraction (`testDocUploadPipelineSuccess`).
3. **Valid DOCX OOXML Pipeline**: Tested genuinely valid OOXML `.docx` (ZIP containing `word/document.xml`) upload, storage, metadata persistence, and Apache Tika text extraction (`testDocxUploadPipelineSuccess`).
4. **Note Attachment Pipeline**: Tested lecture note creation with optional PDF attachment (`testUploadNoteWithAttachmentSuccess`).

---

38. SWAGGER/OPENAPI DOCUMENTATION

All Week 5 endpoints (`/api/teacher/upload-pdf`, `/api/teacher/upload-note`, `/api/documents/{id}/download`, `/api/documents/{id}/stream`) are fully documented with Swagger UI annotations (`@Tag`, `@Operation`, `@Parameter`).

---

39. FILES CREATED

1. `FileStorageService.java`
2. `ProcessingStatus.java`
3. `UploadedDocument.java`
4. `Notes.java`
5. `UploadedDocumentRepository.java`
6. `NotesRepository.java`
7. `DocumentTextExtractionService.java`
8. `DocumentDTO.java`
9. `CreateNoteRequest.java`
10. `NoteDTO.java`
11. `DocumentService.java`
12. `NotesService.java`
13. `TeacherDocumentUploadController.java`
14. `DocumentDownloadController.java`
15. `DocumentIngestionIntegrationTest.java`
16. `WEEK_5_IMPLEMENTATION_REPORT.md`
17. `WEEK_5_IMPLEMENTATION_REPORT.pdf`
18. `build_week5_report_pdf.py`

---

40. FILES MODIFIED

1. `application.yml` (Added `file.upload-dir`, `file.max-size-bytes`, and `spring.servlet.multipart` settings)
2. `SecurityConfig.java` (Updated `/api/teacher/**` matcher to allow `hasAnyRole("TEACHER", "ADMIN")`)

---

41. REQUIREMENT-BY-REQUIREMENT CHECKLIST

Mentor Requirement	Target Day	Status	Empirical Evidence
Implement local physical file storage service	Day 25	**Completed**	`FileStorageService.java` created with safe path resolution.
Configurable upload directory (`uploads/`)	Day 25	**Completed**	Configured in `application.yml`.
Enforce safe file naming strategy	Day 25	**Completed**	`UUID + "_" + sanitizedFilename` implemented.
Prevent path traversal attacks	Day 25	**Completed**	`startsWith(uploadPath)` check enforced and tested.
Validate extensions & MIME (PDF, DOC, DOCX)	Day 25	**Completed**	Extension and MIME validation enforced.
Enforce maximum file size (20 MB)	Day 25	**Completed**	Enforced in `FileStorageService` and `spring.servlet.multipart`.
Implement `UploadedDocument` JPA entity	Day 26	**Completed**	Entity created with metadata fields and `extractedText`.
Implement `Notes` JPA entity	Day 26	**Completed**	Entity created with teacher, subject, chapter, and document link.
Enforce authenticated teacher ownership	Day 26	**Completed**	Derived strictly from SecurityContext principal.
Implement `UploadedDocumentRepository` & `NotesRepository`	Day 26	**Completed**	Repositories created in `com.learnpulse.backend.repository`.
Implement `POST /api/teacher/upload-pdf`	Day 27	**Completed**	RestController endpoint created and verified.
Implement `POST /api/teacher/upload-note`	Day 27	**Completed**	RestController endpoint created and verified.
Implement `GET /api/documents/{id}/download`	Day 28	**Completed**	Download controller endpoint created and verified.
Implement `GET /api/documents/{id}/stream`	Day 28	**Completed**	Streaming controller endpoint created and verified.
Integrate Apache Tika & PDFBox text extraction	Day 29	**Completed**	`DocumentTextExtractionService.java` created and verified.
Perform comprehensive end-to-end automated testing	Day 30	**Completed**	Executed 29/29 automated tests (`mvn clean test` **`BUILD SUCCESS`**).

42. ITEMS INTENTIONALLY NOT IMPLEMENTED

In strict compliance with the mentor's Week 5 specification, the following AI and business modules were **intentionally NOT implemented**:

- **RAG Subsystem**: No document passage chunking, chunk overlap logic, or retrieval pipelines.
- **Embeddings Engine**: No vector embedding generation (e.g. OpenAI/Spring AI/SentenceTransformers).
- **Vector Search**: No `pgvector` similarity queries or cosine distance indexing.
- **LLM Integration & AI Tutor**: No prompt construction, LLM API calls, streaming response generation, or AI chat interfaces.
- **Assessment & Quiz Subsystem**: No quiz generation, question creation, or student progress tracking.

43. KNOWN LIMITATIONS

1. **OCR for Scanned PDF Images**: Apache PDFBox extracts text from native digital PDFs. Scanned image-only PDFs require an external OCR engine (such as Tesseract), which belongs to optional future enhancements.

44. WEEK 5 SUMMARY

Week 5 has successfully delivered a robust, secure, and production-ready **Content Upload Engine & Local Storage Provider** for LearnPulse AI. The system safely ingests educational documents (PDF, DOC, DOCX), enforces path traversal security and file size limits, extracts plain text via Apache PDFBox and Apache Tika, persists metadata to PostgreSQL, and provides secure document download and streaming APIs.

All features have been verified via 29 automated unit and integration tests, achieving **100% build success (`BUILD SUCCESS`)** and zero test failures.

> ***"Only the functionality specified in the Week 5 mentor specification was implemented. No RAG, embeddings, vector search, LLM, AI Tutor, or other future-week functionality was implemented."***

End of Official Week 5 Implementation Report.